

React Function Component Hands-On Lab — Solution

Building the Student Management Portal Score Calculator ("scorecalculatorapp")

Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class components
- Explain function components
- Define the component constructor
- Define the render() function

In this hands-on lab, you will learn how to:

- Create a function component
- Apply style to components
- Render a component

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

Create a React app for a Student Management Portal named "scorecalculatorapp" and create a function component named "CalculateScore" which accepts Name, School, Total, and Goal (the number of subjects/tests) in order to calculate and display the average score of a student.

Solution — Step by Step

Step 1: Create the React project "scorecalculatorapp"

Open a terminal in Visual Studio Code and run the following command to scaffold the project:

```
npx create-react-app scorecalculatorapp
```

Once the setup finishes, navigate into the project folder:

```
cd scorecalculatorapp
```

Step 2: Create a Components folder and the CalculateScore file

Under the src folder, create a new folder named "components". Inside it, add a new file named CalculateScore.js.

Step 3: Write the CalculateScore function component

Add the following code in CalculateScore.js. It accepts name, school, total, and goal as props: goal represents the number of subjects/tests, so the average is calculated as $\text{total} \div \text{goal}$.

```
import React from 'react';
import '../stylesheets/mystyle.css';

function CalculateScore(props) {
  const average = (props.total / props.goal).toFixed(2);

  return (
    <div className="score-card">
      <h1>Student Score Calculator</h1>
      <p><strong>Name:</strong> {props.name}</p>
      <p><strong>School:</strong> {props.school}</p>
      <p><strong>Total Marks:</strong> {props.total}</p>
      <p><strong>Number of Subjects:</strong> {props.goal}</p>
      <p><strong>Average Score:</strong> {average}</p>
    </div>
  );
}

export default CalculateScore;
```

Step 4: Create a Stylesheets folder and mystyle.css

Under the src folder, create a new folder named "stylesheets" and add a file named mystyle.css to style the component:

```
.score-card {
  max-width: 400px;
  margin: 40px auto;
  padding: 20px 30px;
  border: 2px solid #4CAF50;
  border-radius: 10px;
  background-color: #f4fff4;
  font-family: Arial, sans-serif;
  text-align: left;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.15);
}

.score-card h1 {
  color: #2e7d32;
  text-align: center;
  font-size: 22px;
}

.score-card p {
  font-size: 16px;
  margin: 8px 0;
}
```

Step 5: Edit App.js to invoke the CalculateScore component

Update App.js to import CalculateScore and render it, passing name, school, total, and goal as props:

```
import React from 'react';
import './App.css';
import CalculateScore from './components/CalculateScore';

function App() {
  return (
    <div className="App">
      <CalculateScore
        name="Ananya Sharma"
        school="Greenfield Public School"
        total={450}
        goal={5}
      />
    </div>
  );
}

export default App;
```

Step 6: Run the application

In the command prompt, navigate into the scorecalculatorapp folder (if not already there) and run:

```
cd scorecalculatorapp
npm start
```

Step 7: View the application in the browser

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

The page will display the styled score card showing the student's Name, School, Total Marks, Number of Subjects, and the calculated Average Score, confirming that the CalculateScore function component is rendering and styled correctly.

Result

A React application named "scorecalculatorapp" was successfully created with a function component named CalculateScore that accepts Name, School, Total, and Goal as props, computes the average score ($\text{Total} \div \text{Goal}$), and displays it. The component was styled using an external mystyle.css file, rendered from App.js, and verified by running the app locally and viewing it in the browser at <http://localhost:3000>.